TECNOLOGIE E ARCHITETTURE PER LA GESTIONE DEI DATI
A.A. 2024/2025

MODELLAMENTO DI SISTEMI: ASPETTI QUALITATIVI

Capitolo 2

P. ATZENI, L. BELLOMARINI, S. IANNUCCI

INTRODUZIONE

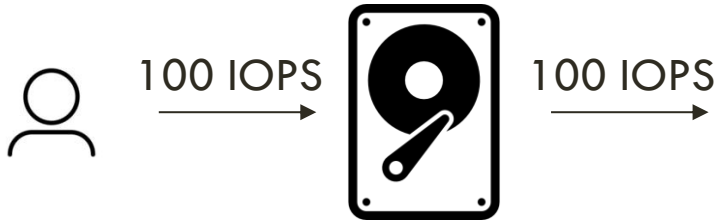
Abbiamo visto come sia importante *progettare le prestazioni* di un sistema

Introduciamo un *framework* per questo scopo, basato sull'idea che i sistemi di computer:

- Sono composti da risorse (e.g., processori, dischi, canali di comunicazione, thread di processi, sezioni critiche, lock su database, …)
- Le risorse sono in qualche modo condivise dalle varie richieste a cui il sistema è soggetto

Le richieste inviate al sistema competono per le risorse e possono crearsi delle *code*

- Per questo motivo, il *framework* che useremo sarà basato su *modelli di code (queuing models)*
- Poiché ogni risorsa ha una propria coda, parleremo di *reti di code (queuing networks)*



Utilizzazione = 100%

Utilizzazione = 100%

MODELLAMENTO

Modello di un sistema: astrazione di un sistema reale.

- Il livello di dettaglio e gli aspetti da considerare dipendono dallo scopo del modello.
  Esempio: se il sistema eseguirà esclusivamente applicazioni CPU-intensive, ***potrebbe*** essere non necessario modellare le prestazioni dei dischi
- L'unico modello di un sistema completamente affidabile è una copia del sistema stesso!
  - Molto costoso; difficile, se non impossibile da gestire

DUE TIPOLOGIE DI MODELLI

Modelli Simulativi:

- *B*asati su un programma che simula i diversi componenti del sistema. Il workload può essere *riprodotto (replayed)* da una traccia o da un benchmark o può essere generato secondo qualche distribuzione di probabilità.
- I componenti modellati sono arricchiti con *contatori* per metriche di prestazione, che possono essere usati al termine della simulazione per calcolare statistiche. Ad esempio, il calcolo del tempo di risposta medio di un componente può essere ottenuto con: $T = \frac{\sum_{i=1}^{n_t} T_i}{n_t}$, dove T è il tempo di risposta medio, T_i è il tempo di risposta della richiesta i e n_t è il numero totale di richieste.
- Possono essere usati per condurre degli studi molto dettagliati perché si basano su un alto livello di dettaglio per la loro realizzazione
- A causa dell'alto livello di dettaglio, possono essere molto costosi e difficili da realizzare.

DUE TIPOLOGIE DI MODELLI (2)

Modelli Analitici

Costituiti da un insieme di formule e/o algoritmi che forniscono i valori delle misure di prestazioni desiderate in funzione del workload.

- Sono solitamente meno dettagliati dei modelli simulativi, al fine di renderli matematicamente trattabili
- Come conseguenza, possono essere meno accurati dei modelli simulativi…
- … ma anche più efficienti da eseguire.

DUE TIPOLOGIE DI MODELLI (3)

Quindi, in sintesi:

- I modelli analitici sono meno costosi da costruire e tendono ad essere computazionalmente più efficienti dei modelli simulativi
- A causa del maggior livello di astrazione dei modelli analitici, ottenere i parametri di input è più semplice che nei modelli simulativi
- I modelli simulativi possono essere arbitrariamente dettagliati e possono essere più accurati dei modelli analitici
- Alcuni comportamenti non possono essere gestiti dai modelli analitici (non esistono formule in forma chiusa) e necessitano quindi dei modelli simulativi

In pratica, spesso si usano entrambi. Ad esempio, si può partire con un modello simulativo (che sia correttamente implementato!) per verificare se i risultati prodotti da un dato modello analitico sono soddisfacenti. Se lo sono, si butta via il simulativo e si continua con l'analitico, sapendo che si sta introducendo un certo margine d'errore.

Nel *capacity planning* non cerchiamo la precisione assoluta: risultati che hanno un errore fino al 30% sono generalmente accettabili.

In questo corso adotteremo un approccio ibrido: studieremo i modelli analitici, ma spesso li risolveremo con un simulatore.

ESEMPIO: UN SEMPLICE DB SERVER

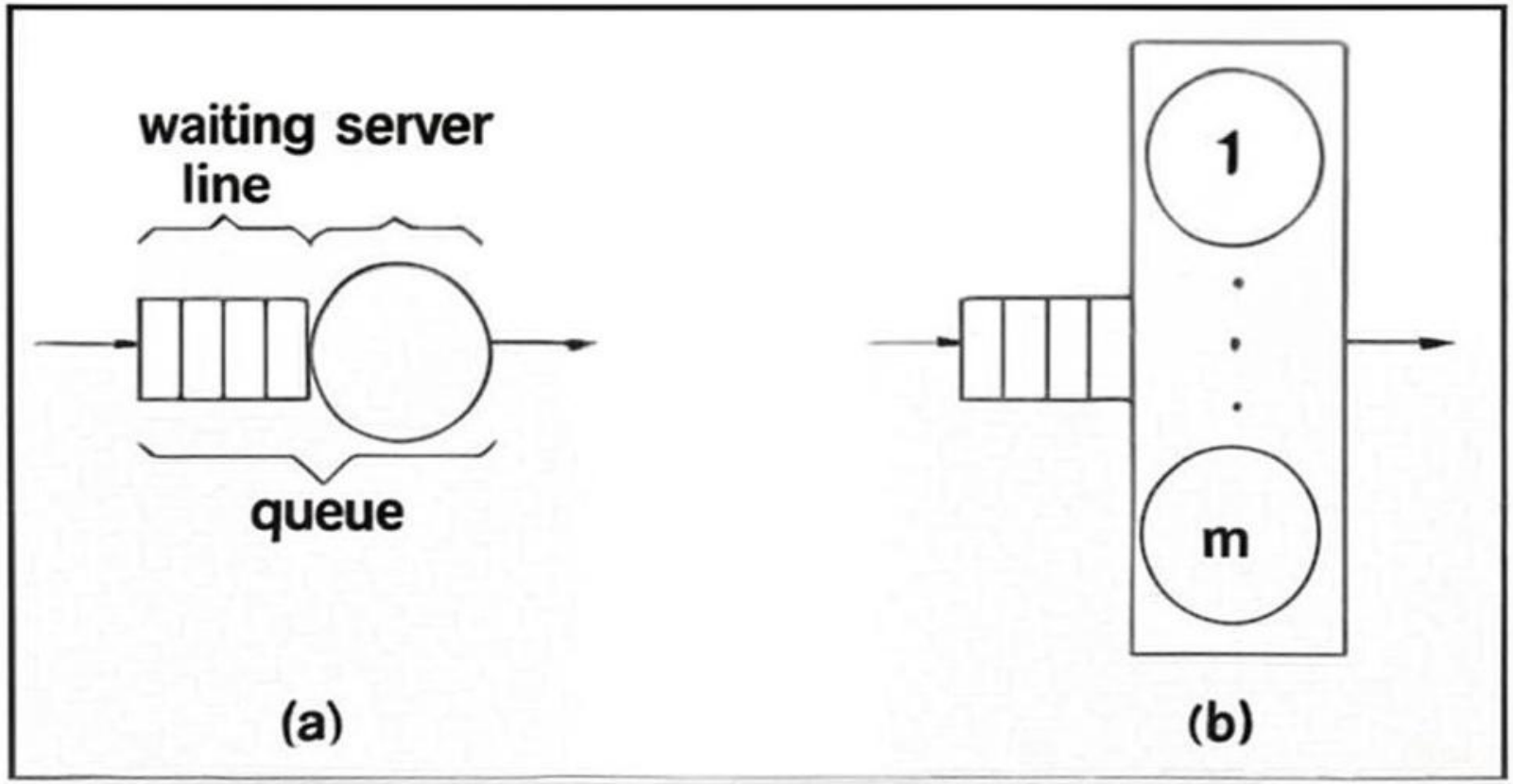
Consideriamo un DB server caratterizzato da:

- 1 CPU e 1 disco
- Un tasso di arrivo delle transazioni pari a 1.5 transazioni al secondo (tps)

Normalmente, durante l'esecuzione di una transazione vengono utilizzate in modo alternato le risorse di CPU e disco. Molto spesso più di una volta ciascuna.

- Quindi, in un dato istante di tempo, potremmo avere: due transazioni in fase di processamento nel sistema - una che sta utilizzando la CPU, l'altra che sta utilizzando il disco – ed altre transazioni in attesa di essere processate da una delle due risorse
- E' ragionevole quindi immaginare le risorse come dei "centri di calcolo" ai quali si accede dopo aver fatto un po' di coda

CODE



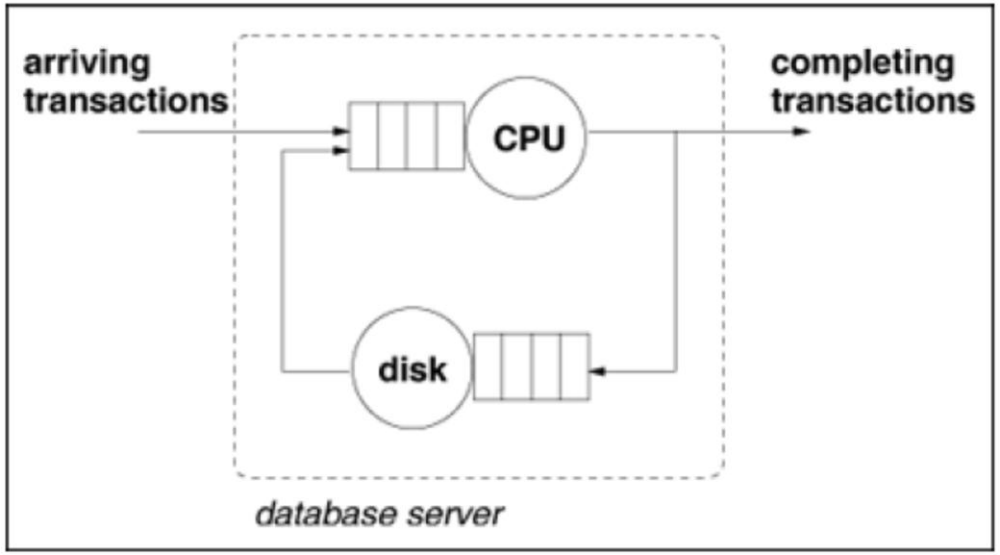
MODELLAMENTO DI UN SEMPLICE DB SERVER

Con la notazione che abbiamo appena visto, è possibile realizzare lo scheletro del nostro primo modello di DB Server come una rete di code (QN)!

Mappare un sistema esistente ad un QN non è banale. Potremmo ad esempio voler conoscere:

- Come varia il tempo di risposta del DB ad una transazione in funzione del tasso di arrivi a cui è soggetto
- Come cambia il tempo di risposta del DB ad una transazione se viene sostituita la CPU (e/o il disco)

A seconda delle domande che vogliamo fare, dobbiamo modellare più o meno aspetti del sistema reale



CLASSI MULTIPLE

Supponiamo che dai file di log del database ci rendiamo conto che esistono transazioni con caratteristiche profondamente diverse. Supponiamo di poterle raggruppare (per semplicità in questo esempio) in tre classi: *semplici, medie, complesse*.

- Ad esempio, si potrebbe utilizzare una combinazione del tempo di CPU ed del numero di operazioni di I/O per transazione per fare questa divisione

Ragionare per *classi di servizio* ci consente di ottenere delle approssimazioni molto migliori nel modello. Ad esempio, potremmo ottenere:

Classe di servizio	Percentuale sul totale	Tempo medio di CPU (sec)	Numero medio di I/O
Semplice	45%	0.04	5.5
Medio	25%	0.18	28.9
Complesso	30%	1.20	85.0

CLASSI MULTIPLE (2)

Un modello QN multi-classe dovrebbe essere usato nei seguenti casi:

- Valori eterogenei di service demand, come nel caso descritto nella tabella precedente
  - **Service demand: tempo medio totale speso da una transazione in una data risorsa**
- Tipi diversi di workload
  - Esempio: il DB server considerato potrebbe essere usato sia per brevi transazioni online che per processamenti batch (ad esempio per la generazione di report).
- Diversi Service Level Objectives (SLO). Classi di servizio diverse possono avere differenti SLO associati.
  - Esempio: nella tabella precedente, potremmo voler definire dei limiti superiori per il tempo di risposta pari a 1.2 secondi per la classe "semplice", 2.5 secondi per la classe "medio", 8 secondi per la classe "complesso".

CLASSI APERTE, CLASSI CHIUSE

Classe di servizio	Percentuale sul totale
Semplice	45%
Medio	25%
Complesso	30%

Negli esempi delle slide precedenti, l'intensità del workload era specificato in *tps*. Supponendo un tasso di arrivo complessivo pari a 1.5 tps:

- Il tasso di arrivo per la classe "semplice" è $1.5 \times 0.45 = 0.675$ tps
- Il tasso di arrivo per la classe "medio" è $1.5 \times 0.25 = 0.375\ tps$
- Il tasso di arrivo per la classe "complesso" è $1.5 \times 0.3 = 0.45\ tps$

Una classe di workload specificata in questo modo è detta ***classe aperta*** ed ha le seguenti caratteristiche:

- Intensità di workload specificato tramite un tasso di arrivi. Questo valore è solitamente indipendente dallo stato del sistema (i.e., dal numero di transazioni che il sistema sta già gestendo)
- Numero di clienti nel sistema potenzialmente *illimitato*.
- *Throughput* fornito come parametro di input. In equilibrio, il throughput di una classe aperta è uguale al suo tasso di arrivo, ed è quindi noto.

CLASSI APERTE, CLASSI CHIUSE (2)

Consideriamo ora lo scenario in cui, di notte, il DB non è disponibile per transazioni online ma solo per dei job batch per la produzione di report

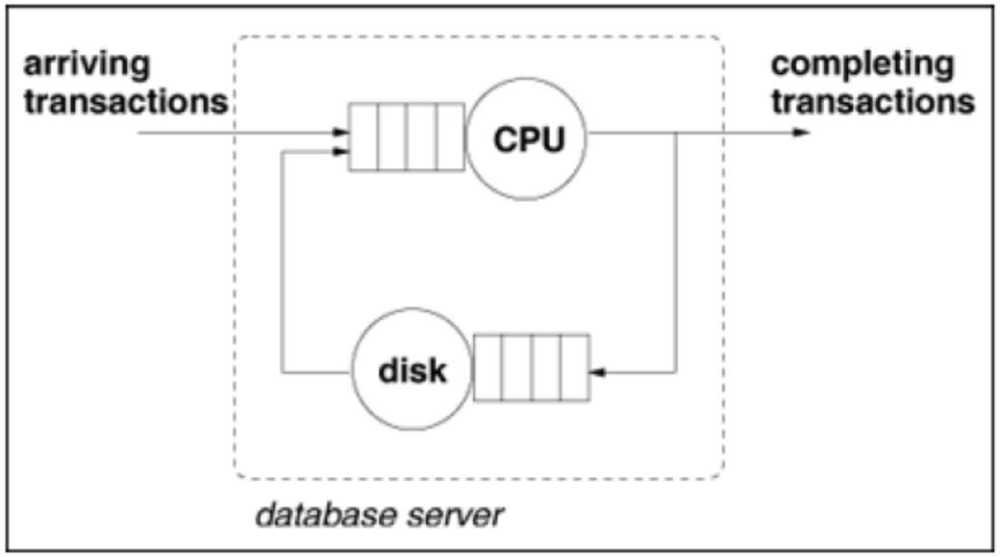
Una classe di workload di questo tipo è chiamata *chiusa* ed è caratterizzata da:

- Intensità di workload specificata dalla popolazione all'interno del sistema
  - Esempio: numero di richieste concorrenti per l'esecuzione dei job batch pari a *5*.
    Supponendo di partire con un numero di job da eseguire maggiore di *5*, si mandano i primi *5* in esecuzione. Al completamento del primo, uno di quelli in coda entra, e così via, mantenendo un numero di job attivi all'interno del sistema grosso modo pari a *5*.
- Numero di clienti del sistema limitato e conosciuto
- Throughput è un parametro di output. Si ottiene risolvendo il modello QN.

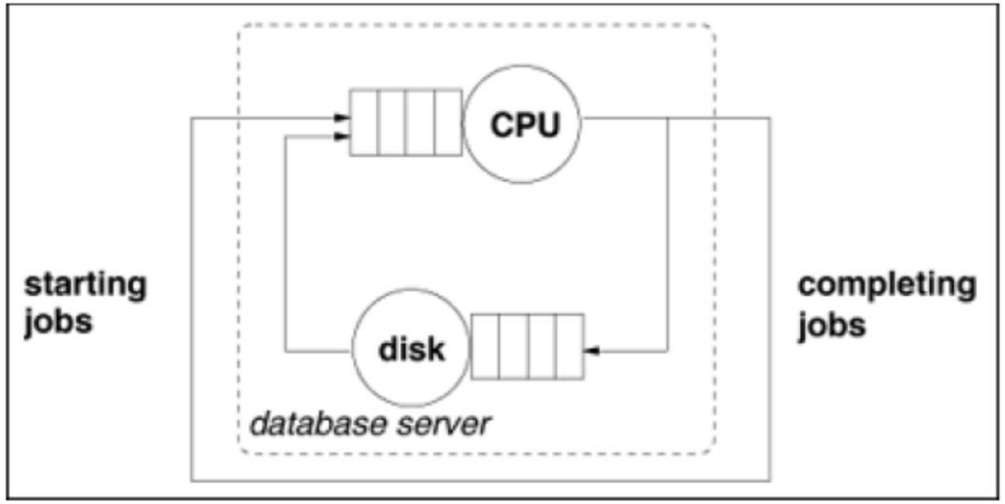
Un modello QN in cui tutte le classi sono chiuse è detto *chiuso*

MODELLO QN APERTO VS MODELLO QN CHIUSO

Modello QN Aperto



Modello QN Chiuso



MODELLO MISTO

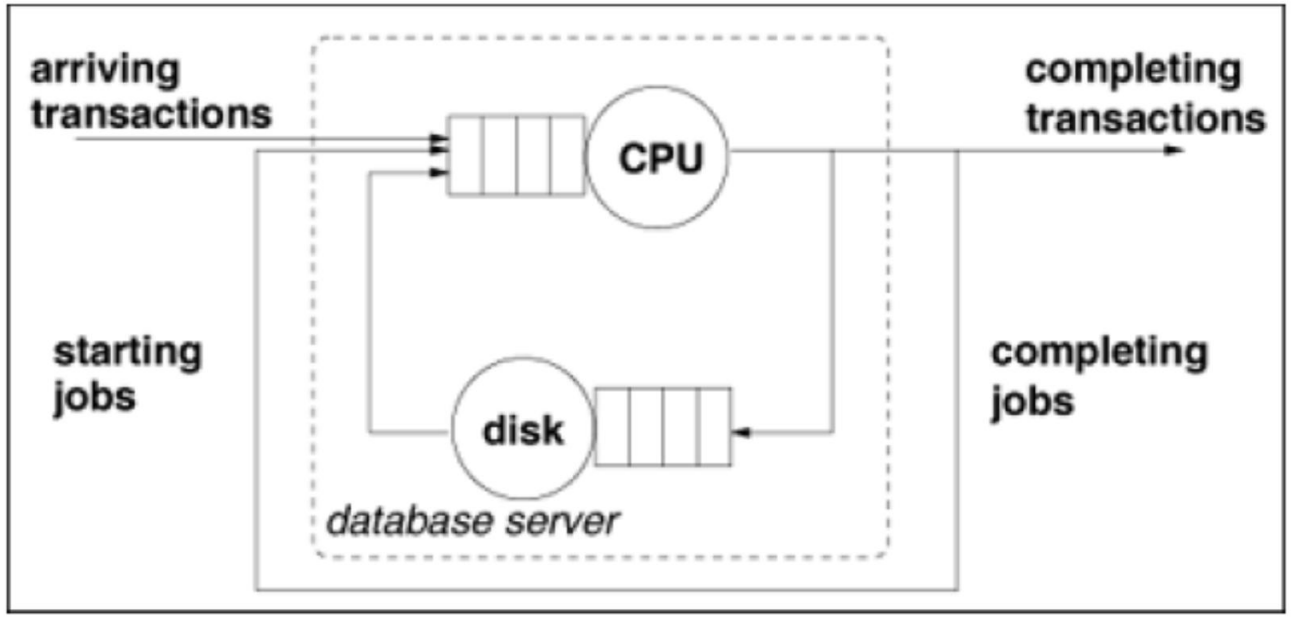
Consideriamo il DB Server analizzato precedentemente.

Si vuole sapere se si riescono a rispettare gli SLO eseguendo insieme: il workload delle classi aperte e quello delle classi chiuse

Classe di servizio	Tempo medio di risposta massimo (sec)	Throughput minimo
Semplice	1.2	-
Medio	2.5	-
Complesso	8.0	-
Batch Reports	-	20/ora

MODELLO MISTO (2)

Il modello necessario per rispondere a tale domanda è un modello misto, perché alcune classi sono aperte, altre chiuse



SERVICE LEVEL AGREEMENT

Gli obiettivi di prestazioni specificati nel precedente caso sono anche chiamati *Service Level Agreement (SLA)*, e sono dei veri e propri contratti che possono essere stipulati tra fornitore di servizi e cliente, riguardanti requisiti di qualità

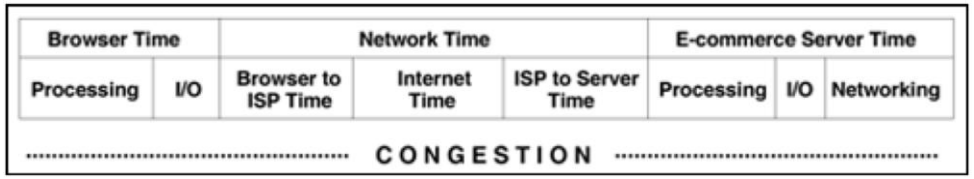
Ad esempio, il fornitore di un sito web potrebbe stabilire con i suoi clienti il seguente SLA:

- 99.99% di disponibilità tra le 8 e le 23, 99.9% nel resto del tempo;
- Max 4 secondi per il tempo di scaricamento di una pagina per tutte le richieste inviate su connessioni non sicure, e max 6 secondi per richieste inviate su connessioni sicure
- Minimo throughput di 2000 pagine al secondo
- Una certa penale nel caso di non soddisfacimento dei requisiti di qualità

RISORSA DI TIPO RITARDO (DELAY)

Supponiamo di voler rappresentare l'interazione di un umano con un sito web:

- L'umano clicca su un link, la richiesta fa il percorso che abbiamo visto nella precedente lezione e la pagina viene caricata



- L'umano legge il contenuto (ed impiega del tempo per farlo) e poi, nel caso, segue un altro link, e così via.

In altre parole, l'umano introduce un *ritardo (delay)*, dovuto alla lettura, tra una richiesta e la successiva.

- Questo ritardo può essere modellato come un centro senza coda, detto appunto *delay*.

RISORSA DIPENDENTE DAL CARICO *(LOAD-DEPENDENT)*

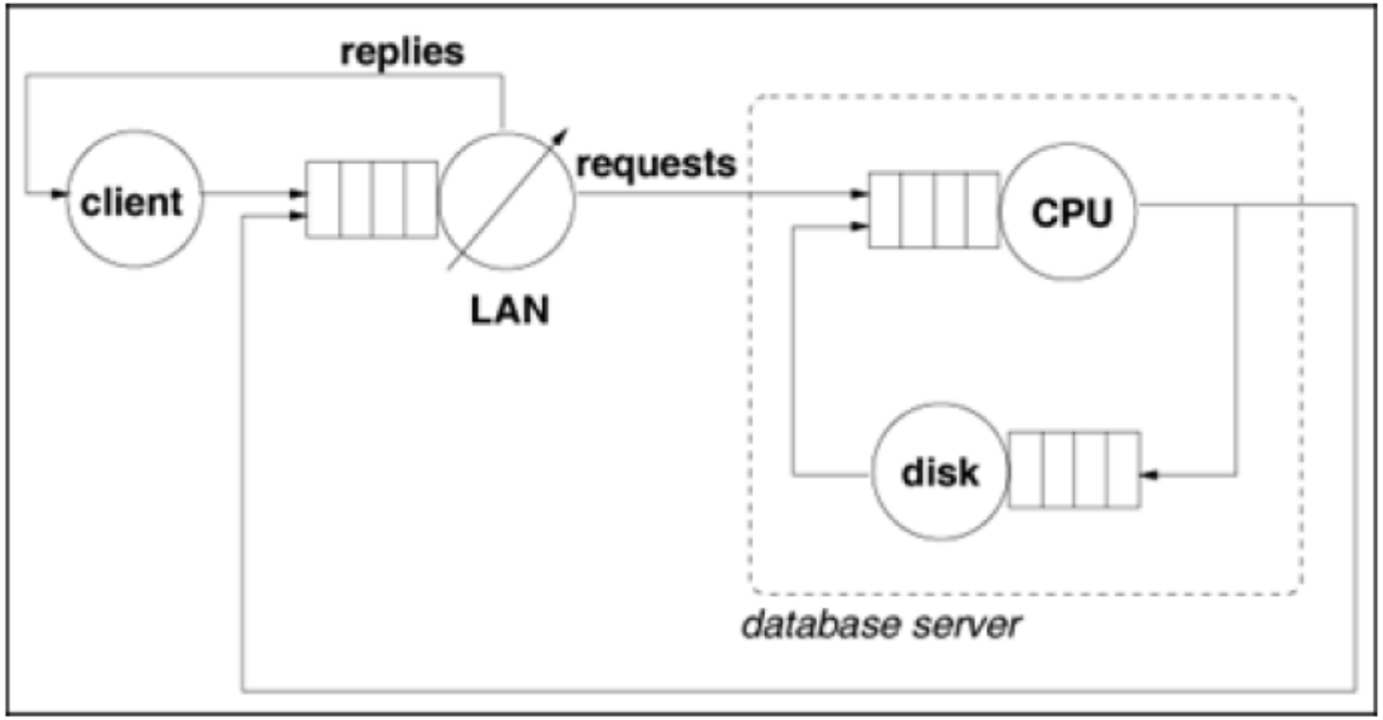
Sempre con riferimento all'esempio di prima, supponiamo ora che il computer che sta navigando su web sia collegato su una LAN insieme a molti altri computer.

- E' ragionevole pensare che la velocità di trasmissione *percepita* dal computer in questione dipenda dal traffico generato dagli altri computer, data una certa ampiezza di banda condivisa.

In altre parole, la velocità della rete che stiamo considerando *dipende* dal carico al quale è al momento sottoposta

Una risorsa di questo tipo è modellabile tramite un centro *load-dependent*.

QN PER DATABASE CON CLIENT E LAN



TIPI DI RISORSE

Per riassumere, possiamo usare nei modelli QN tre diversi tipi di risorse:

- Load Independent. Queste risorse hanno un tasso di servizio che non dipende dal carico o dal numero di richieste in coda
- Load dependent. Il tasso di servizio di questo tipo di risorse è una funzione del numero di richieste in coda.
- Delay. In questo caso non c'è coda. Una richiesta che arriva ad una risorsa di tipo delay è servita immediatamente. Questo tipo di risorsa è solitamente usata per modellare componenti dedicati, che non vengono cioè condivisi con altre richieste, o situazioni in cui il numero di risorse appartenenti al sistema è molto maggiore alle richieste, e quindi non si verificano mai code.

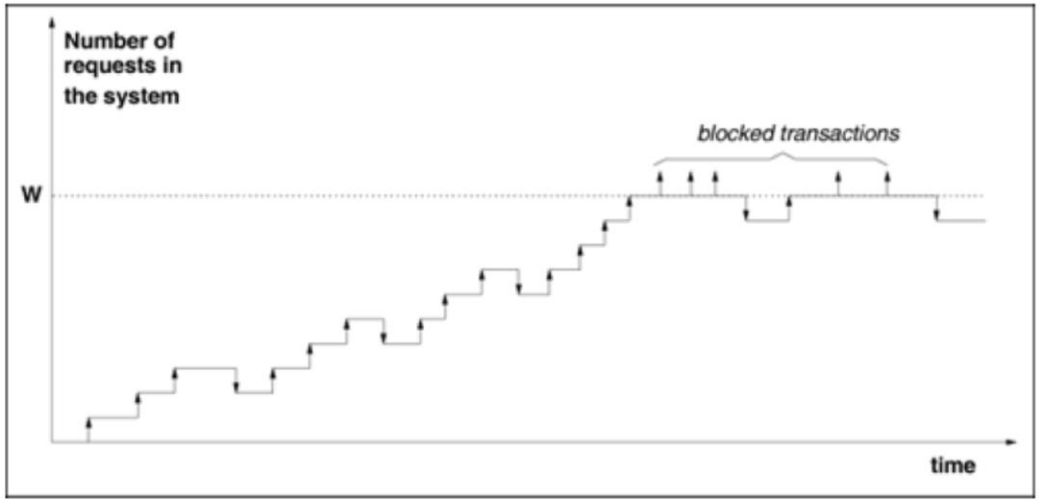
ADMISSION CONTROL

Supponiamo di voler offrire qualche tipo di garanzia sulle prestazioni del server DB che gestiamo. Non importa quale hardware stiamo usando, esisterà sempre un valore massimo W di transazioni concorrenti che il sistema può gestire.

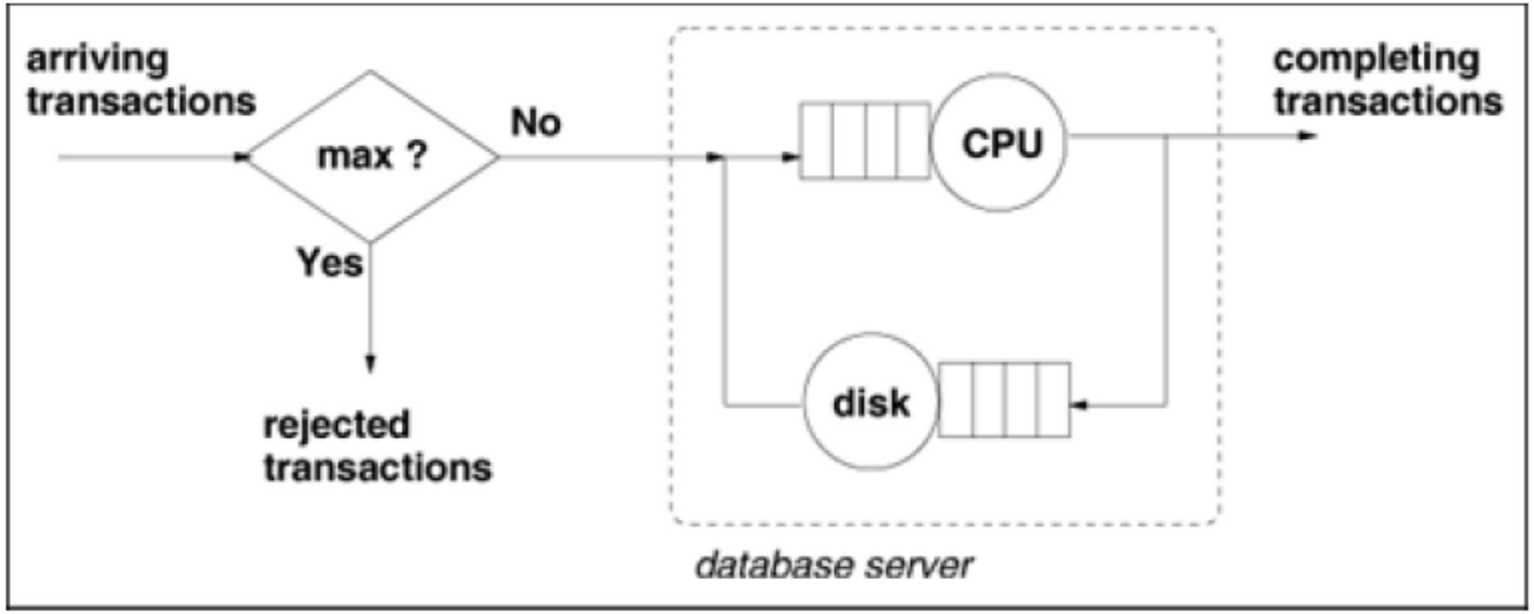
- Solitamente è un parametro facilmente gestibile da file di configurazione del DB server.
- In postgres, parametro *max_connections* in *postgresql.conf* (di default impostato a 100)

In caso di admission control, il throughput non è necessariamente uguale al tasso di arrivo, e può essere calcolato con:

Throughput=
Arrival Rate x (1 – Prob of Rejection)



QN CON ADMISSION CONTROL



DISCIPLINE DI ACCODAMENTO

Ogni centro, a meno che non sia un *delay*, ha una coda. Ma come si accodano le richieste?

- First Come First Served (FCFS). Le richieste vengono servite nell'ordine di arrivo alla coda
- Priority Queuing. Quando il servente diventa disponibile, il job con la priorità più alta viene servito. Se c'è più di una richiesta a priorità massima, viene tra questi applicata FCFS.
  Esistono moltissime varianti: priorità *statiche* e *dinamiche;* utilizzo o meno di *prelazione*; prelazione con *continuamento (preemptive resume)* o con *ripartenza (preemptive restart)*, …
- Round Robin (RR). In questo caso, ogni transazione è servita per un breve periodo di tempo, comunemente chiamato *quanto* o *slice (fetta) temporale*, dopo il quale la risorsa passa alla prossima transazione, rimettendo in coda la precedente in un modo circolare.
- Processor Sharing (PS). E' una RR portata all'estremo, con un quanto di tempo talmente piccolo che l'impressione è quella che più richieste siano in esecuzione contemporaneamente all'interno della risorsa.
- Altre discipline: SJF, MLFQ, …